B.Tech | Machine Learning | Case Study

Fake Product Review Detection

Using NLP and Random Forest

A. Lakshmi Sai Teja
Roll No: 2520080005

D. Vignesh Kumar
Roll No: 2520090071

P. Vedadri
Roll No: 2520090060

Guide: Dr. Y. Lakshmi Prasanna, Assistant Professor, CSIT

Institution: Koneru Lakshmaiah Education Foundation (KLEF)

Academic Year: 2026–2027 | Department of Computer Science and Information Technology

Abstract

Fake reviews mislead shoppers, damage trust in online marketplaces and unfairly reward dishonest sellers. This project builds a supervised classifier that labels a product review as Genuine or Fake.

A labelled dataset of 500 reviews (18 columns) was explored, 13 numerical features were selected, and a Random Forest (200 trees) was trained on an 80/20 stratified split. A Flask web app lets users test reviews and view dataset and model analytics.

The model scored 100% on the 100 held-out reviews. We treat this cautiously: the data is small, repetitive and highly separable.

Fake review detection | Random Forest | Binary classification | EDA | Feature importance

500
labelled reviews

13
numerical features

100%
test score (cautious)

Introduction

1

Reviews drive buying

Shoppers use star ratings and reviews as a trusted shortcut, so reviews strongly influence sales.

2

Fake reviews distort trust

Promotional, defamatory and bot-written reviews inflate or damage ratings and are often short, repetitive and over-enthusiastic.

3

ML can learn the pattern

Labelled examples let a classifier learn which review characteristics separate fake from genuine.

Domain: Machine Learning for e-commerce trust & safety **Task:** classify a review as Genuine (0) or Fake (1)

Problem Statement

Case-study scenario

A marketplace lists 100 products and receives hundreds of reviews monthly. The trust-and-safety team sees clusters of five-star reviews with identical wording, many exclamation marks and non-verified buyers. They need a lightweight model to score each review and prioritise inspection.

Challenges

- Manual moderation cannot keep up with volume
- Fake reviews imitate genuine ones
- Spammers adapt their tactics over time

Formal definition

$$f(x) \rightarrow y$$

$x \in \mathbb{R}^{13}$: 13 numerical features per review

$y \in \{0, 1\}$: 0 = Genuine, 1 = Fake

Binary classification, judged on held-out data

Who is affected?

Shoppers | **Honest sellers** | **Platforms** | **Moderators**

Central question: can a Random Forest on review metadata and simple text statistics reliably separate fake from genuine reviews, and which features drive the decision?

Motivation

1

Real-world need

Fake reviews mislead customers, erode trust and penalise honest sellers.

2

Scale problem

Reviews arrive far faster than people can moderate; a scoring tool can direct humans to the most suspicious ones.

3

Explainable detector

A simple, data-driven model shows which review traits give fakes away.

4

Complete ML practice

Covers data, EDA, encoding, training, evaluation, interpretation and a working web app.

Objectives

Main objective: build and evaluate a Random Forest classifier that separates fake from genuine product reviews and explains why.

1. Understand the structure, quality and class balance of the dataset
2. Identify which review traits differ most between fake and genuine
3. Encode categorical fields and select 13 numerical features
4. Train a Random Forest on a stratified 80/20 split
5. Evaluate with accuracy, precision, recall, F1, ROC-AUC, PR-AUC and confusion matrix
6. Interpret results with feature importance and expose them in a Flask web app

Existing System

Current approaches

Manual moderation

Trust-and-safety analysts read and judge reviews

Rule / keyword filters

Blocklists of phrases, spam words, repeated posts

Surface signals

Average stars, verified-buyer badges, helpful votes

Limitations

- Manual review does not scale with review volume
- Fixed rules are easy for spammers to evade
- Single signals (stars, votes) are weak on their own
- No ranking of which reviews to inspect first
- Little explanation of why a review looks suspicious

Proposed System

A feature-based Random Forest detector with a Flask web interface that scores a review from 13 numerical features.

Does not scale	Automated scoring of every review in milliseconds
Easy to evade rules	Learns patterns from labelled data; combines many signals
Weak single signals	13 features combined by 200 decision trees
No explanation	Gini feature importance shows which traits matter
No easy demo	Web app: enter features, get Genuine / Fake verdict

System Architecture

OFFLINE TRAINING PIPELINE

Dataset
reviews.csv 500 rows × 18 columns

→

Preprocess
Encode Yes/No and label; pick 13 features

→

Split
Stratified 80/20: 400 train, 100 test

→

Random Forest
200 trees, class_weight balanced

→

Evaluate + save
Metrics, confusion matrix, saved model

ONLINE PREDICTION (WEB APP)

User input
13 feature values entered in the form

→

Flask backend
Validates input, builds feature vector

→

Saved model
Random Forest loaded for scoring

→

Prediction
Genuine or Fake result card

→

Analytics pages
Dataset EDA and model performance views

Methodology / Workflow

1

Load data

Read CSV with pandas; confirm 500 × 18

2

Inspect

Types, missing values (none), unique values

3

EDA

Class balance, ratings, verified status

4

Encode

Yes/No → 1/0; Genuine/Fake → 0/1

5

Select features

13 numerical columns as X, label as y

6

Split

80/20 stratified, random_state = 42

7

Train

RandomForestClassifier, 200 trees

8

Evaluate + interpret

Metrics, matrix, importance, web app

Implemented Classifier vs. Broader NLP Pipeline

IMPLEMENTED & EVALUATED

- 13 numerical features (rating, verified, votes, text statistics, sentiment ratios, duplicates)
- Random Forest, 200 trees, 80/20 stratified split
- Flask web app: Detect, Dataset, Model Performance pages
- All reported metrics come from this model

DESCRIBED IN ABSTRACT / PROPOSED EXTENSION

- Text cleaning, stop-word removal, lemmatization
- TF-IDF vectors on review text
- Logistic Regression (L1/L2) baseline
- PCA + DBSCAN bot-cluster detection
- Bayesian hyper-parameter search
- FastAPI REST deployment and drift monitoring

In the app, Review Text and Product Name are display-only; the model never sees the text itself.

Technology Stack

Language

Python 3.8+

ML & data

scikit-learn (RandomForestClassifier), pandas, NumPy

Visualisation

Matplotlib / Seaborn for EDA, ROC and PR figures

Web app

Flask backend, Bootstrap 5 front end

Data store

CSV file (reviews.csv, 500 × 18)

Tools

VS Code and Jupyter Notebook; Windows, macOS or Linux

Proposed extension only: NLTK or spaCy (text preprocessing), FastAPI (REST API)

Dataset Overview

500
Total records

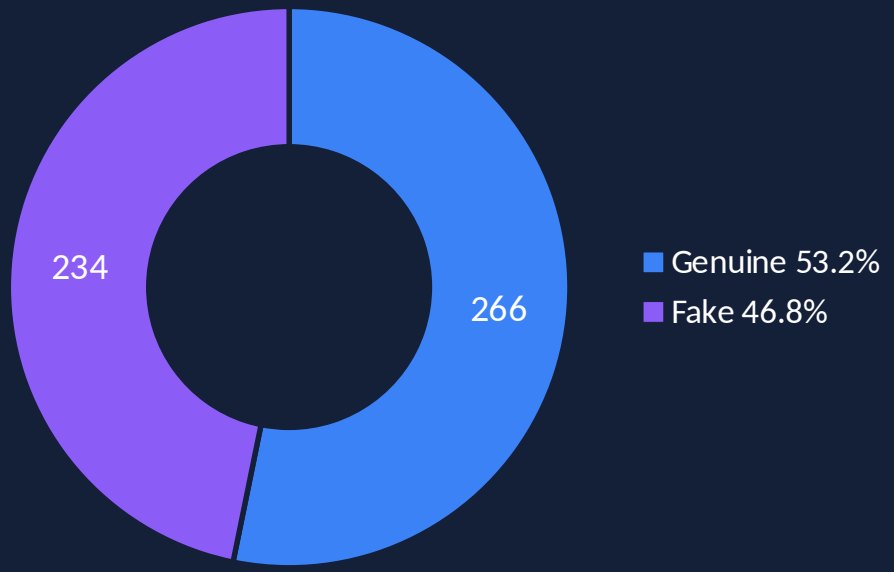
18
Columns

13
Numerical features

400
Training records

100
Testing records

Class distribution (labelled)



Column groups (18 columns)

Identifiers: review_id, product_id

Raw content: review_text, review_date

Metadata: rating, verified_purchase, helpful_votes

Text statistics: word_count, char_count, avg_word_length, exclamation_count, question_count, capital_ratio

Sentiment: positive_word_ratio, negative_word_ratio, sentiment_score

Other: duplicate_count, label (target)

Exploratory Data Analysis: Key Findings

Class-wise feature averages

Feature	Genuine	Fake
helpful_votes	52.8	7.7
word_count	14.2	9.2
avg_word_length	4.43	5.63
exclamation_count	0.00	9.64
capital_ratio	0.027	0.100
positive_word_ratio	0.030	0.309
duplicate_count	0.37	5.48

Source: case-study report EDA.

84.6%

of fake reviews are from unverified buyers; 78.9% of genuine reviews are verified

Data-quality observations

- 0 missing values, 0 duplicate rows
- question_count and negative_word_ratio are all 0
- sentiment_score duplicates positive_word_ratio
- Only 93 unique texts in 500 rows (repetitive)
- Genuine reviews have 0 exclamation marks

Preprocessing & the 13 Model Features

Steps performed

1. Loaded CSV and checked shape (500 × 18)
2. Checked types, missing values, summaries
3. Encoded verified_purchase (Yes=1, No=0)
4. Encoded label (Genuine=0, Fake=1)
5. Selected 13 numeric features; dropped IDs, date, text
6. Stratified 80/20 split, random_state = 42

No scaling or resampling needed

13 numerical features (input vector x)

- rating
- verified_purchase
- helpful_votes
- word_count
- char_count
- avg_word_length
- exclamation_count
- question_count
- capital_ratio
- positive_word_ratio
- negative_word_ratio
- sentiment_score
- duplicate_count

Random Forest Algorithm

How it works



- Each tree learns from a bootstrap sample of the rows
- At each split only a random subset of features is considered
- Final class = majority vote of all 200 trees
- Feature importance = average Gini impurity reduction

Why we chose it

- No feature scaling
- Captures non-linear interactions
- Less overfitting than one tree
- Tolerates redundant features
- Gives feature importance

Implementation

Model settings

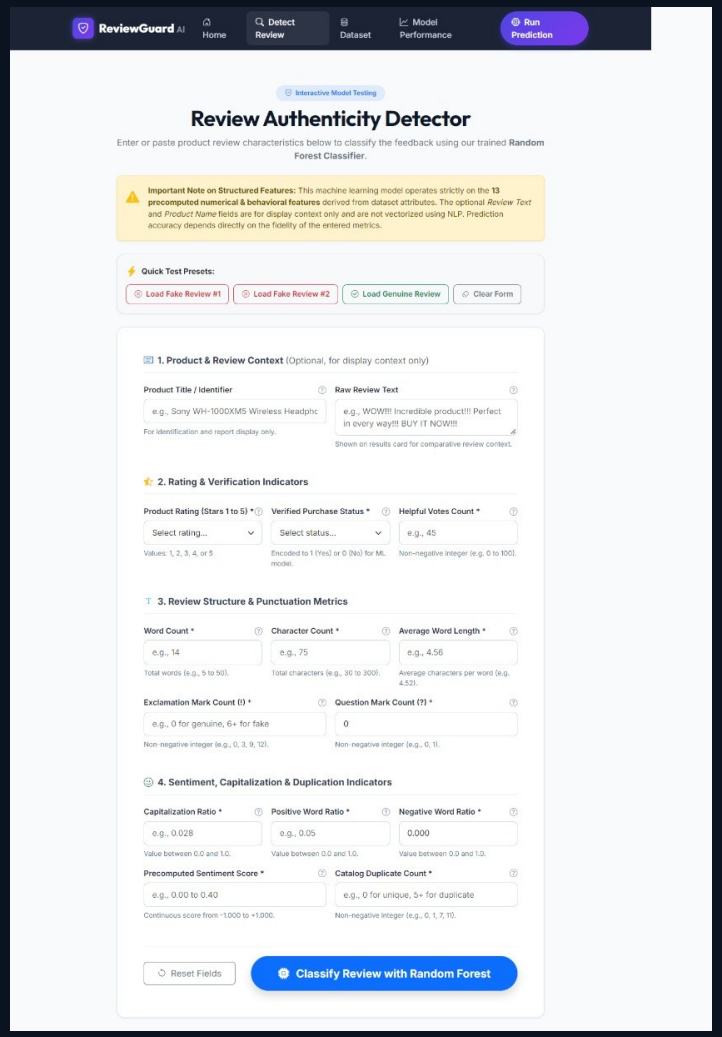
n_estimators	200
max_depth	20
min_samples_split	5
min_samples_leaf	2
class_weight	balanced
random_state	42
Split	400 / 100 stratified

```
# Illustrative training step (scikit-learn)
X = df[FEATURES_13]          # 13 numeric columns
y = df["label_encoded"]      # Genuine=0, Fake=1
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
rf = RandomForestClassifier(n_estimators=200,
    max_depth=20, min_samples_split=5,
    min_samples_leaf=2, class_weight="balanced",
    random_state=42).fit(X_tr, y_tr)
```

After training

- Predicted labels and probabilities on the 100 test rows
- Trained model and encoders saved for reuse by the web app

Web Application: Review Authenticity Detector



How the app works

1 — Enter features

Rating, verified status, helpful votes, word/char counts, exclamation and question counts, ratios, sentiment, duplicates

2 — Quick presets

Load Fake #1, Fake #2 or Genuine sample values, or clear the form

3 — Classify

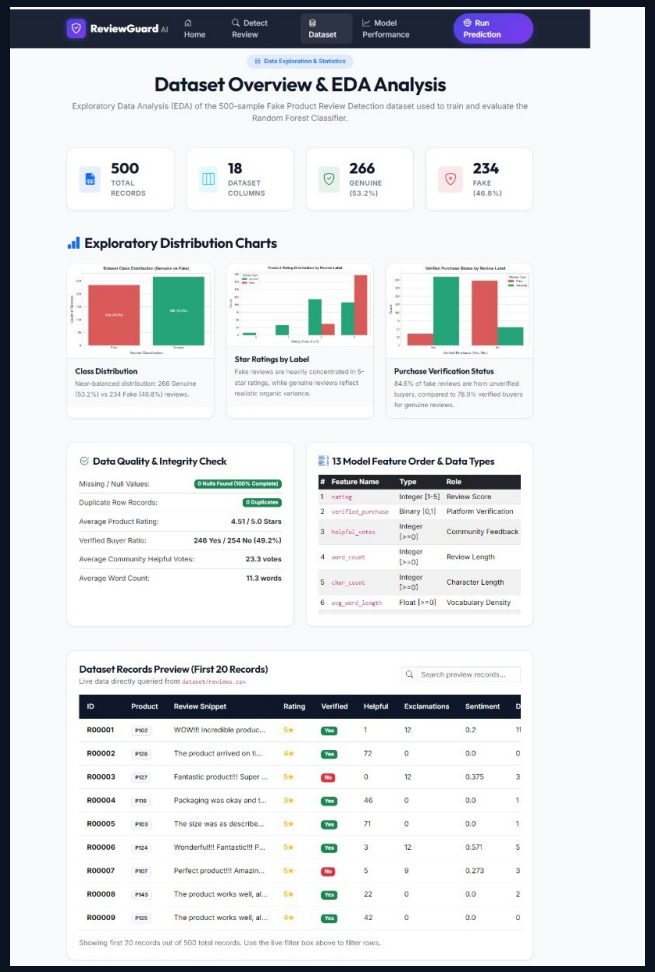
Flask receives the 13 values and passes them to the trained Random Forest

4 — Result card

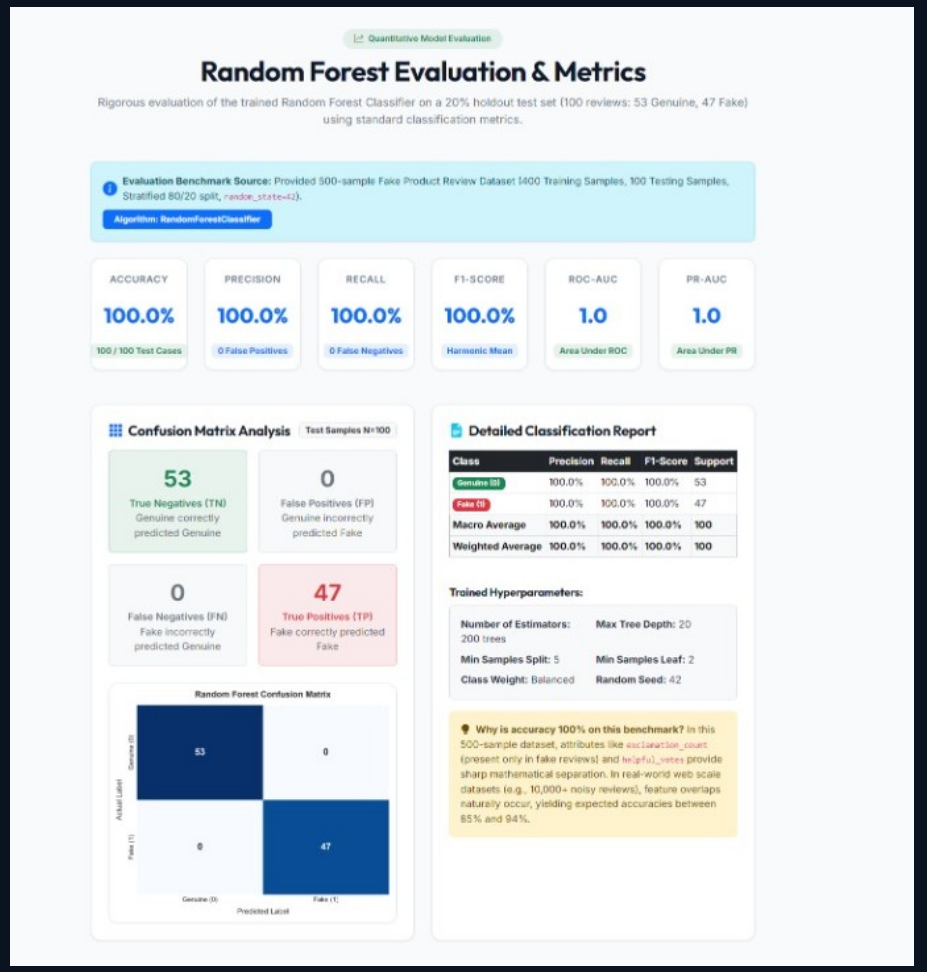
Shows Genuine or Fake; review text and product name are shown for context only

The model uses only the 13 numeric inputs. Accuracy depends on how faithfully they are entered.

Web Application: Dataset & Model Performance Pages



Dataset page



Model Performance page

Dataset page

- KPIs: 500 records, 18 columns
- Class, rating and verification charts
- Quality check and 13-feature table
- Searchable record preview

Model page

- Six metric cards
- Confusion matrix and report
- Hyper-parameters
- Importance, ROC and PR curves

Model Evaluation

100%	100%	100%	100%	1.0	1.0
Accuracy	Precision	Recall	F1-score	ROC-AUC	PR-AUC

Confusion matrix (n = 100)

	Predicted Genuine	Predicted Fake
Actual Genuine	53 TN	0 FP
Actual Fake	0 FN	47 TP

How to read this

- 53 genuine and 47 fake reviews were all classified correctly
- 0 false positives: no genuine review was flagged fake
- 0 false negatives: no fake review was missed
- Test set is small (100) and the data is very separable, so 100% is not a guarantee for real platforms

Feature Importance & Why the Scores Are Perfect

Top Gini importances (from the app)



Why 100%? Read with caution

- Data is small and repetitive (only 93 unique review texts in 500 rows)
- Genuine reviews have 0 exclamation marks; fakes average about 9.6
- Every fake review has duplicate_count above 0
- sentiment_score and positive_word_ratio carry the same information
- Perfect scores may also reflect template-generated data

Next: k-fold cross-validation and realistic data.

Advantages & Applications

Advantages

- Fast, lightweight scoring from 13 numeric features
- Explainable through feature importance
- Works without feature scaling and tolerates redundant features
- Interactive web app for demo, EDA and metrics
- Clear, reproducible workflow (fixed random_state)

Applications

E-commerce marketplaces

Prioritise reviews for moderator inspection

Trust & safety teams

Flag suspicious clusters for human review

Sellers & brands

Monitor suspicious reviews on their listings

Education

Teaching tool for classification and EDA

Use only as a flag for human review, never as the sole basis for action.

Limitations & Future Scope

Limitations

- Only 500 rows; may not generalise to a real marketplace
- Repetitive, partly template-like text (93 unique texts)
- Some features constant or redundant
- Text is not analysed: model uses numeric features only
- Perfect scores come from one small test split
- Spammers adapt, so patterns can be evaded

Future scope

- k-fold cross-validation and a larger, realistic dataset
- Remove redundant features
- Compare with Logistic Regression and gradient boosting
- Add TF-IDF and NLP preprocessing of review text
- PCA + DBSCAN for bot-cluster detection
- REST API deployment and drift monitoring
- Human-in-the-loop moderation workflow

Conclusion

Built

A Random Forest classifier on 13 numerical features with a Flask web app for prediction, EDA and metrics.

Found

Fake reviews are unverified, repetitive, exclamation-heavy, strongly positive and rarely helpful.

Reported

100% accuracy, precision, recall, F1; ROC-AUC and PR-AUC of 1.0 on 100 test reviews.

The result demonstrates the method, not guaranteed real-world performance. Any deployment needs realistic data, cross-validation and a human reviewer in the loop.

References

1. Project documents: Fake Product Review Detection report, case study and abstract, KLEF, 2026–27
2. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32
3. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830
4. Ott, M., Choi, Y., Cardie, C., Hancock, J. (2011). Finding Deceptive Opinion Spam by Any Stretch of the Imagination. ACL
5. Jindal, N., Liu, B. (2008). Opinion Spam and Analysis. WSDM, 219–230
6. Mukherjee, A. et al. (2013). What Yelp Fake Review Filter Might Be Doing? ICWSM
7. Ester, M. et al. (1996). A Density-Based Algorithm for Discovering Clusters (DBSCAN). KDD
8. Documentation: scikit-learn, pandas, Flask, Bootstrap 5